

Inyeccion por comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script root ejecuta algo como 'tar czf backup.tar.gz *' en un directorio, el shell expande el * a la lista de ficheros ANTES de pasarla al comando. Si puedes crear ficheros ahí, creas ficheros cuyo nombre es en realidad una opción del comando (como --checkpoint-action de tar), y el comando ejecuta lo que pongas.

El shell no distingue un fichero llamado '--checkpoint-action=exec=sh script.sh' de una opción legítima: el glob lo expande igual. tar, chown, rsync y otros tienen flags que ejecutan comandos, y así el proceso root corre tu código. Es un privesc-modelo via nombres de fichero.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando cron-abuse o un script root usa un comando con * en un directorio escribible por ti. Muy común con tar en tareas de backup.

Cómo identificarla

- Un cron/script root ejecuta tar/chown/rsync con * en una ruta que puedes escribir.
- El directorio del comodín es de tu propiedad o escribible por tu grupo.
- El comando tiene flags conocidas de ejecución (tar --checkpoint-action).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Crea los ficheros con nombres de flags (--checkpoint=1, --checkpoint-action=...) y espera al siguiente ciclo.

Evidencia esperada: Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh
```

Objetivo y contexto: Prepara el payload que el comando privilegiado acabara ejecutando: copia bash con bit SUID para tener luego una shell root.

Resultado esperado: El fichero shell.sh en el directorio del comodín. Aun no hace nada: falta que el comando root lo invoque via las flags.

Capa 2: prueba documentada

```
touch './--checkpoint=1'; touch './--checkpoint-action=exec=sh shell.sh'
```

Objetivo y contexto: Crea ficheros cuyos nombres son opciones de tar. Cuando el cron ejecute 'tar ... *', el glob los mete como argumentos y tar ejecutara shell.sh como root.

Resultado esperado: Tras correr la tarea, /tmp/rootbash existe con bit SUID: lanzalo con -p para shell root. Ajusta las flags al comando concreto (chown/rsync tienen las suyas).

Anatomía de los payloads

La primera prueba es `echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica el comando con comodín y el directorio (leyendo el cron/script).
- Crea el payload que ejecutara (p. ej. un .sh que ponga SUID a bash).
- Crea ficheros cuyos NOMBRES sean las flags que hacen que el comando ejecute tu payload.
- Espera a que la tarea corra y recoge tu shell/binario root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El comando no usa comodines o los ficheros se listan explícitamente por nombre.

- El directorio del comodin no es escribible por tu usuario ni grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un comando root con * en un directorio donde puedes crear ficheros: nombras ficheros como flags y el shell los expande a argumentos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(wildcard-injection\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.